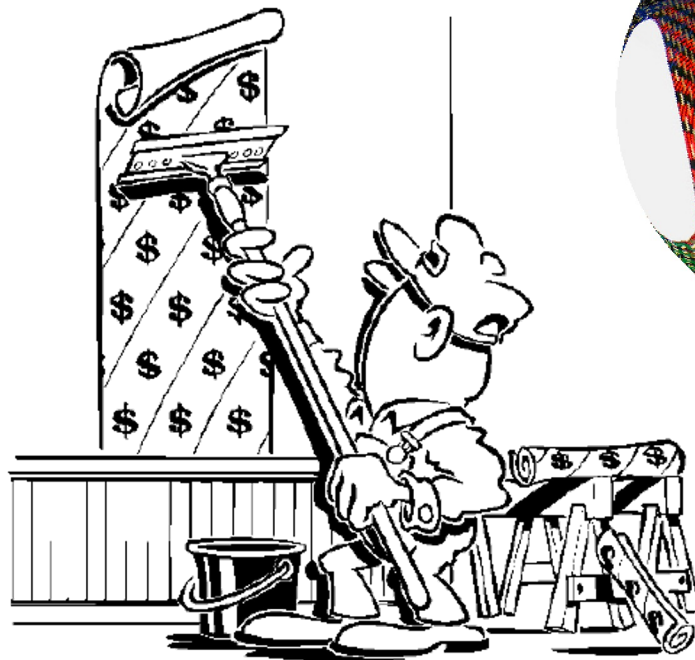


Texture nella Computer Graphics

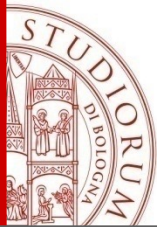


Texture Mapping

Con **Texture** ci si riferisce ad un'immagine di qualsiasi tipo utilizzata per "colorare" la superficie di un oggetto o modello 3D; la superficie del modello apparirà molto più complessa di quello che è, ma soprattutto più realistica.

Con **Texture Mapping** ci si riferisce al processo di applicazione di un'immagine sulla superficie di un modello 3D.





Texture Mapping 2D

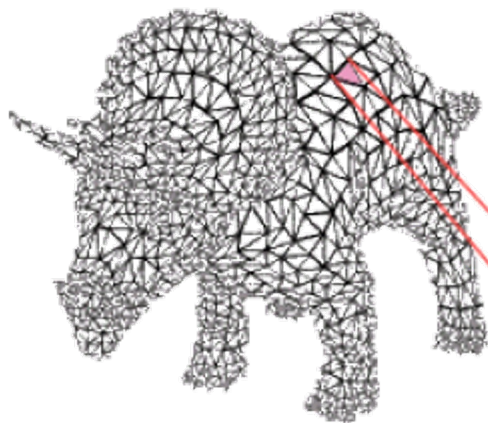
Come si deve procedere ?

Una texture viene definita in un dominio (o spazio texture), mentre l'oggetto è nello spazio 3D; si deve definire una trasformazione fra questi chiamata:

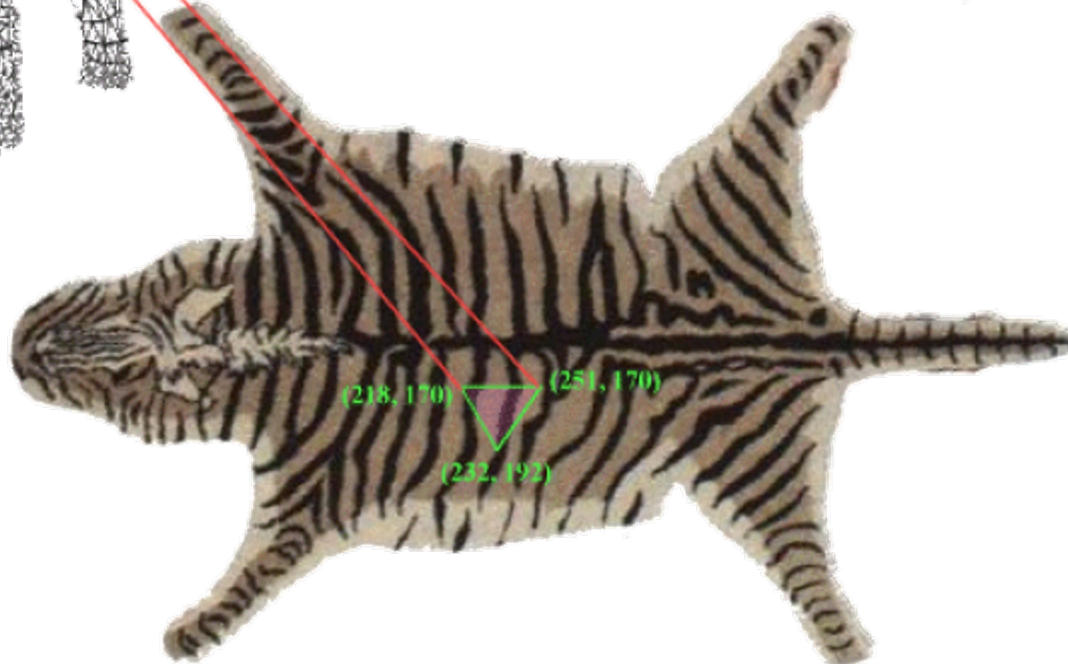
parametrizzazione



Texture Mapping 2D



Ad ogni triangolo nel modello si fa corrispondere una regione triangolare nell'immagine texture.



Esempio di mapping fra spazio texture e mesh 3D. Problema difficile, sia che sia fatto a mano sia in modo automatico.

Texture Mapping 2D: esempio

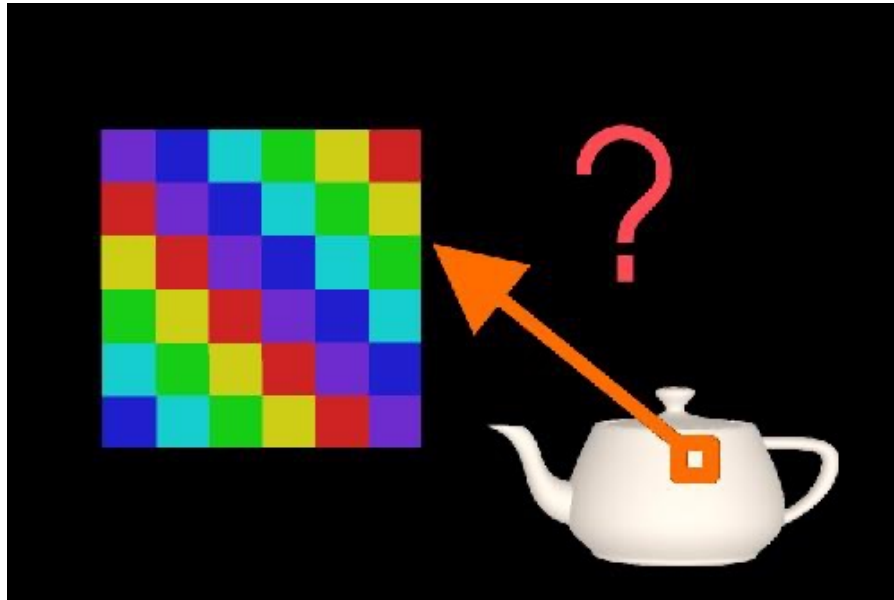
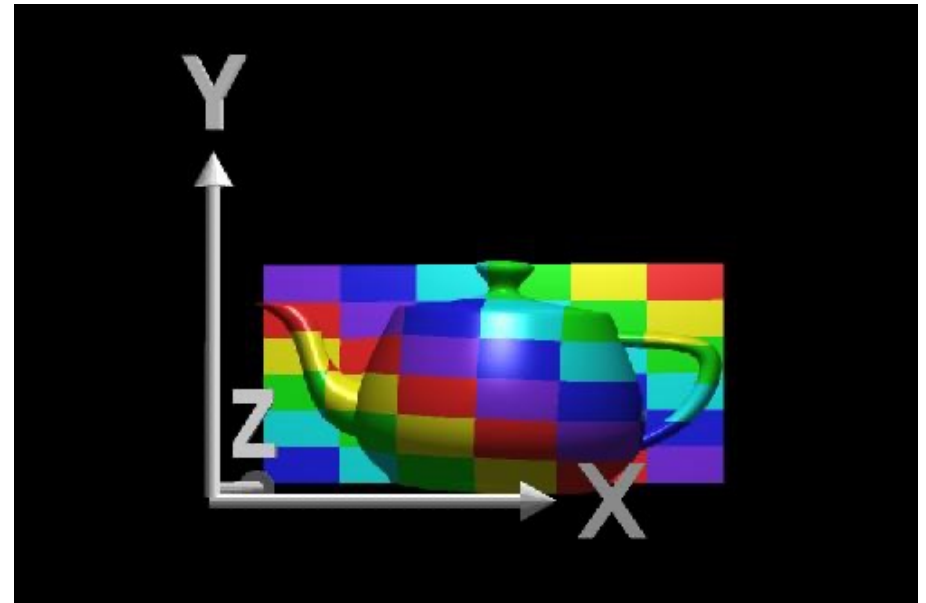


Image Mapping

Parametrizzazione

$$[u, v] = [x, y] \rightarrow [x, y, z]$$



Texture Mapping 2D: esempio

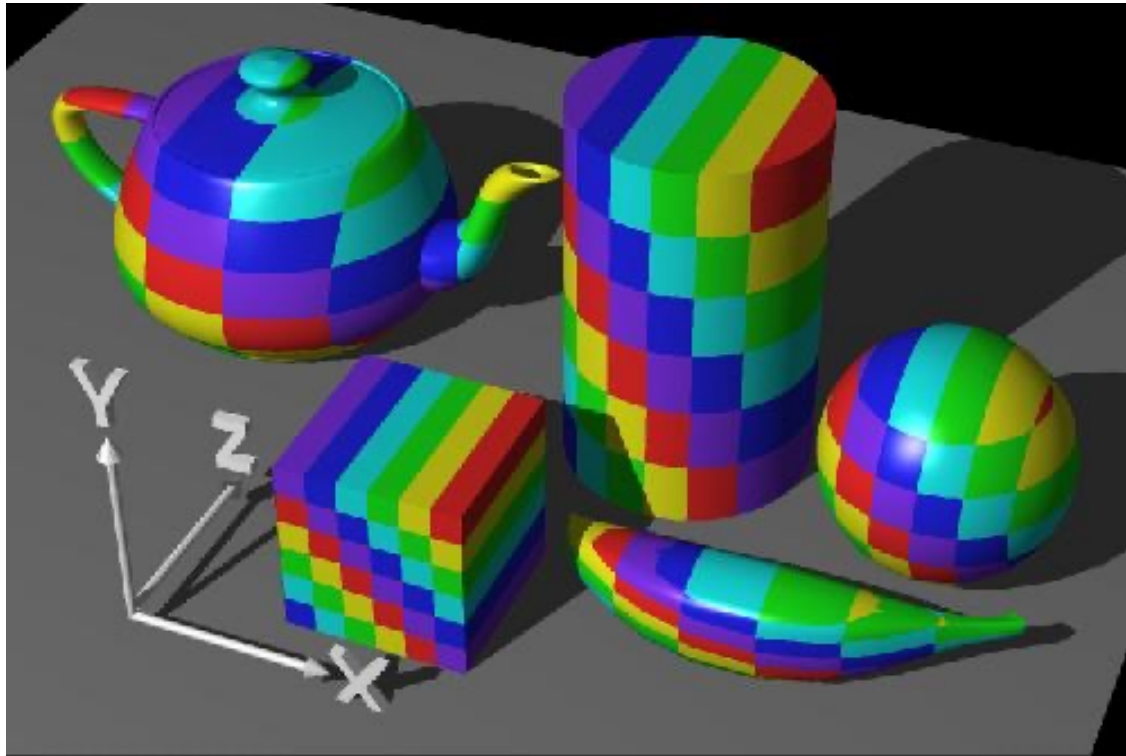
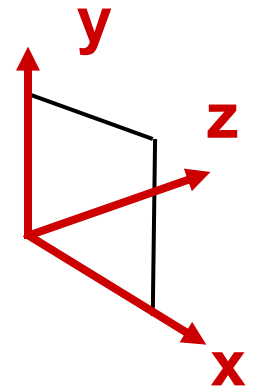


Image Mapping secondo parametrizzazione [u, v] = [x, y] $\rightarrow$ [x, y, z]

Texture Mapping 2D: esempio



parametrizzazione $[u, v] = [x, y] \rightarrow [x, y, z]$



Texture Mapping 2D: esempio

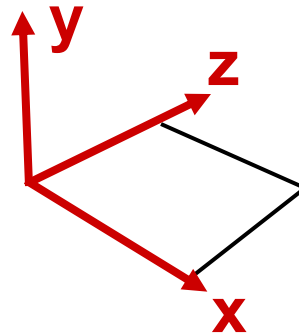
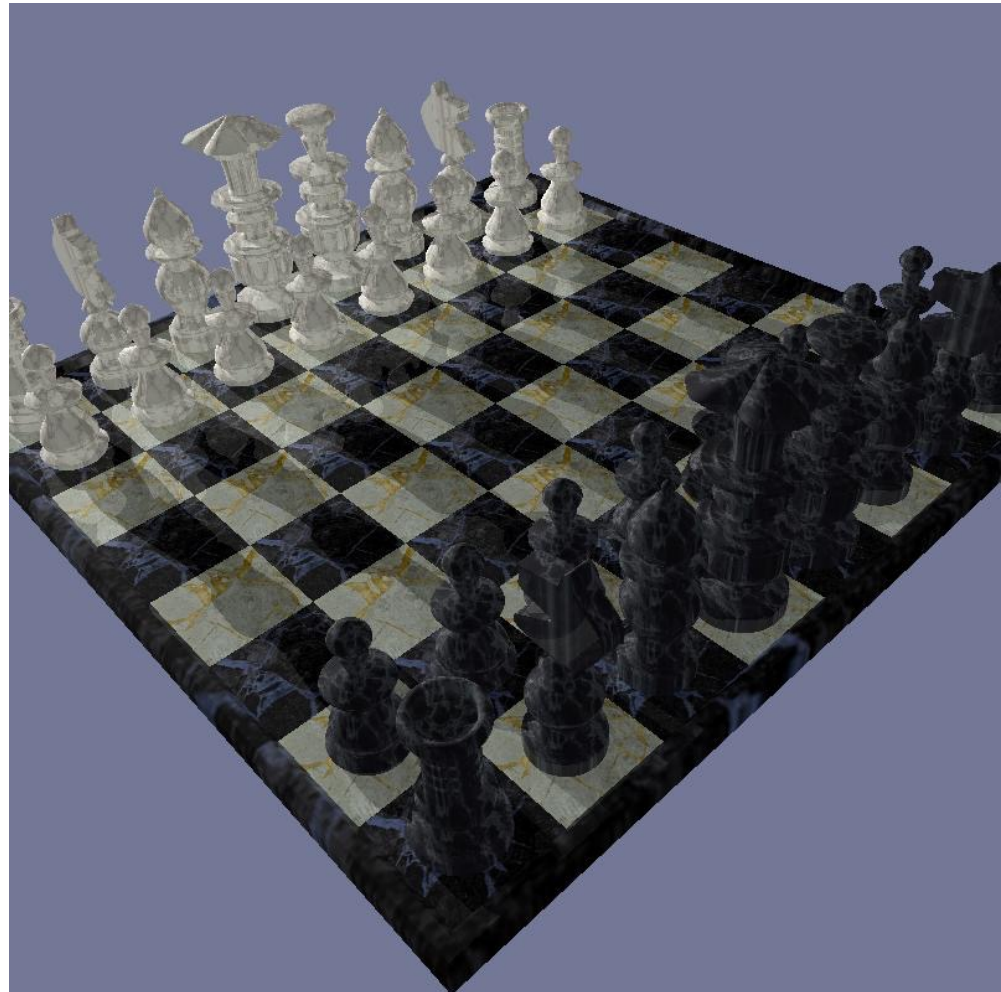
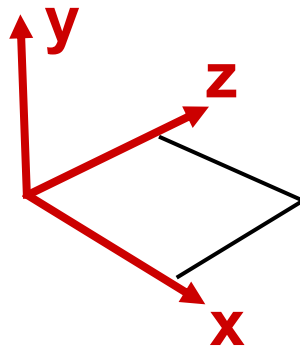


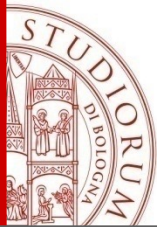
Image Mapping,
parametrizzazione $[u, v] = [x, y] \rightarrow [x, y, z]$
lo spazio texture è stato posizionato alla base dell'oggetto,
quindi il mapping è la proiezione dell'immagine lungo l'asse y .

Texture Mapping 2D: esempio



Texture Mapping 2D: esempio





Texture Mapping 2D

Texture Mapping 2D nella pipeline di rendering.

Dopo la rasterizzazione la palla passa al fragment shader che opera sui singoli fragment/pixel per ogni faccia/triangolo; questo significa che si deve individuare un singolo valore **texture** (**texel**) per ogni fragment/pixel, per essere memorizzato nel frame buffer.

Per fare questo si fa uso di un **Inverse Mapping**; si trova la preimmagine del fragment/pixel corrente nel dominio texture.

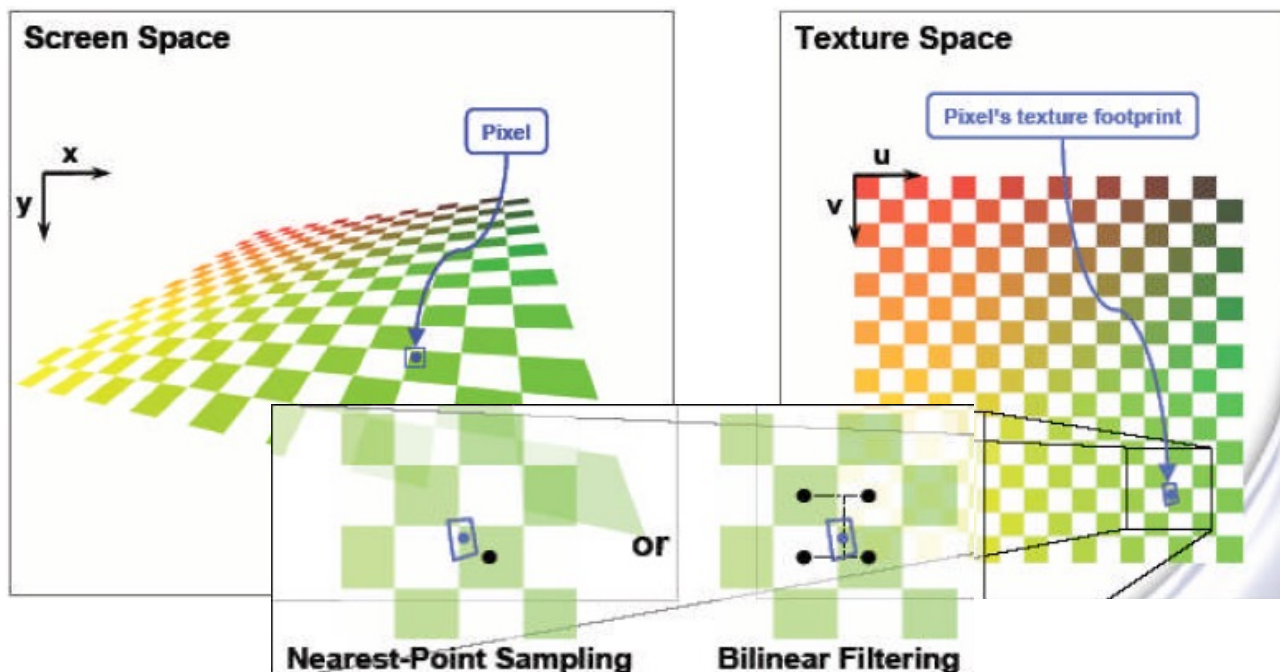
Il texture mapping richiede un trattamento speciale di **Antialiasing** poiché tende a produrre effetti di Aliasing.

Magnification e Minification

Pixel $\rightarrow$ Inverse Mapping $\rightarrow$ Texel

Cosa accade se un texel è più grande di un pixel?

Minification: la texture deve essere rimpicciolita.

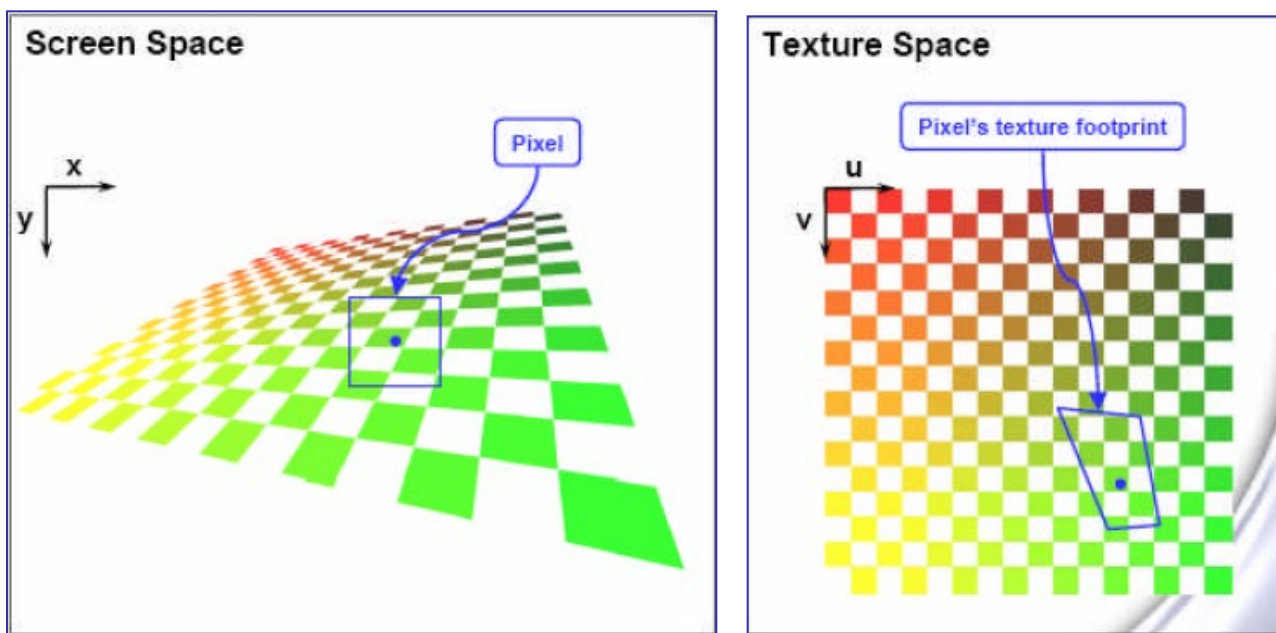


Magnification e Minification

Pixel $\rightarrow$ Inverse Mapping $\rightarrow$ Texel

Cosa accade se un texel è più piccolo di un pixel?

Magnification: la texture deve essere ingrandita.



Magnification e Minification

Tecniche di Antialiasing (o Filtering) più usate:

Nearest Neighbor:

ad ogni pixel viene dato il colore del texel più vicino; più pixel avranno stesso colore



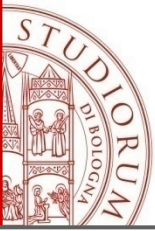
Effetto Pixelation



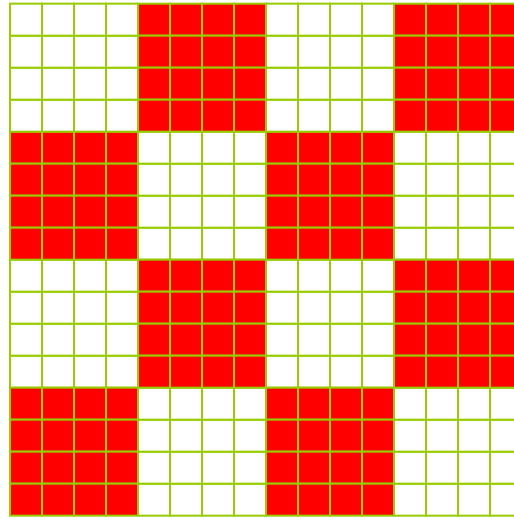
Effetto Blur

Bilinear Interpolation:

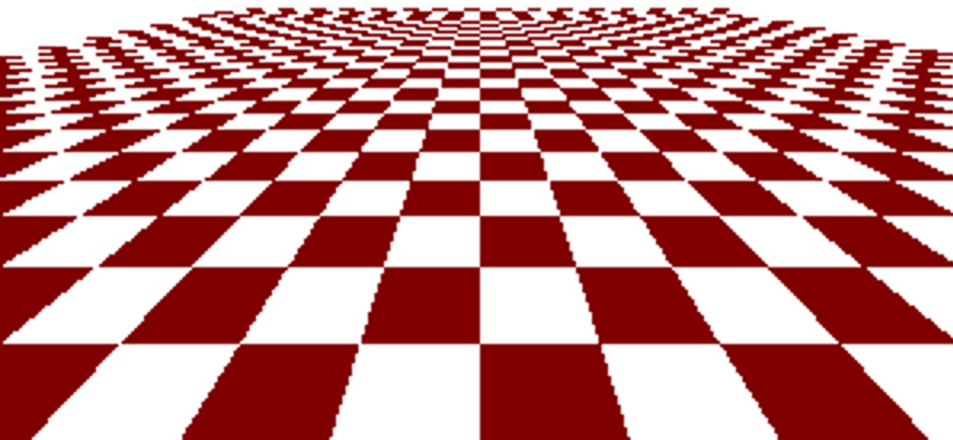
per ogni pixel vengono presi i 4 texel più vicini e interpolati linearmente nelle due dimensioni per avere un valore medio per il pixel



Esempio di Minification

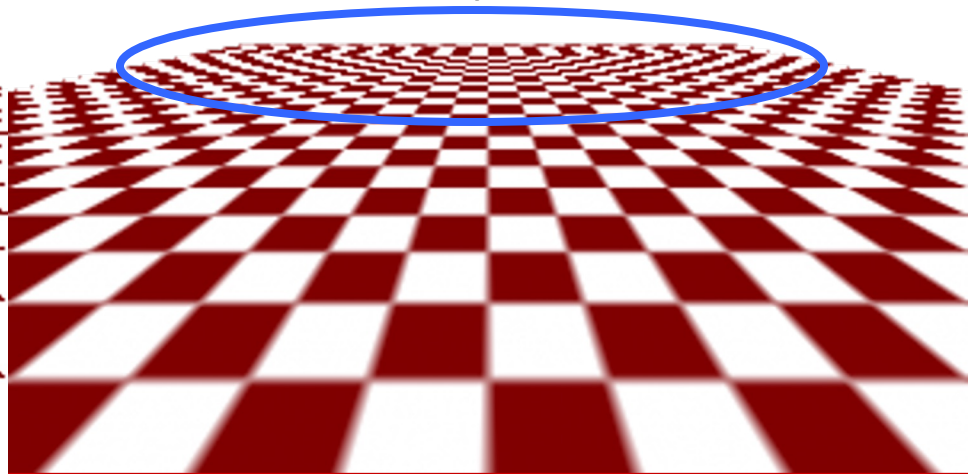


Nearest Neighbor



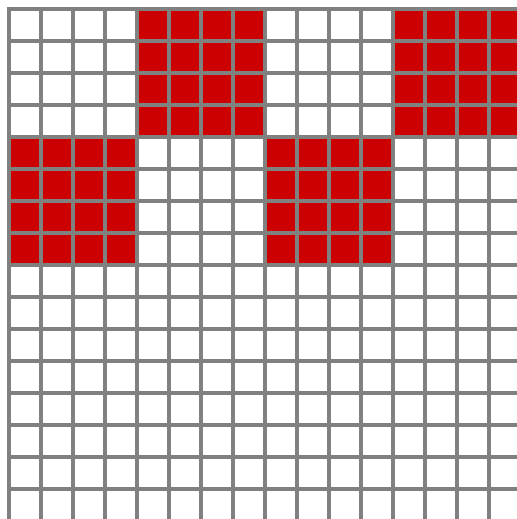
Bilinear Interpolation

non risolve il problema

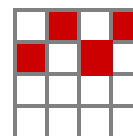
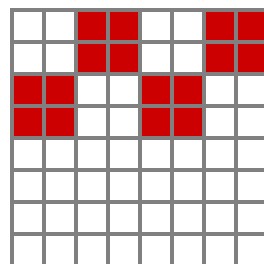


MIP Mapping

Multum In Parvo: molte cose in poco spazio



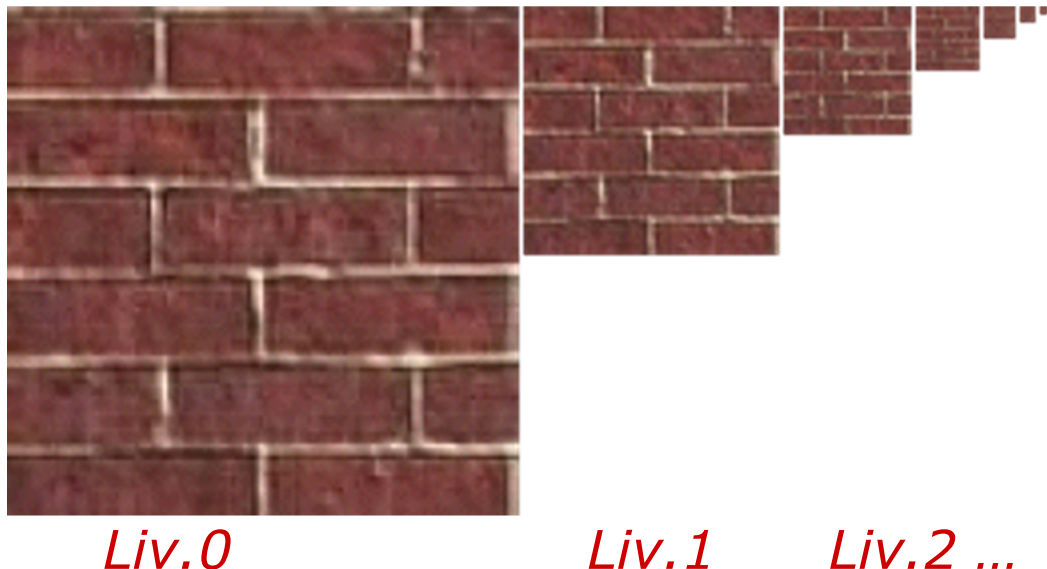
Texture originale



versioni a metà risoluzione

Per una texture di $2^n \times 2^n$ texel, si calcolano $n-1$ texture, ciascuna a metà risoluzione della precedente.

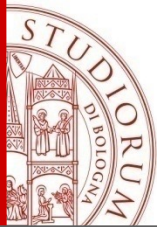
MIP Mapping



Come si genera un MIP-map da una texture?

Per ogni texel al Livello i , si fa la media dei valori dei 4 corrispondenti texel al livello $i-1$.

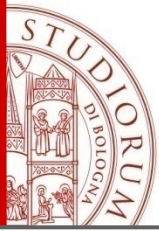
Ogni livello del MIP-map rappresenta una versione pre-blurred di texel multiple



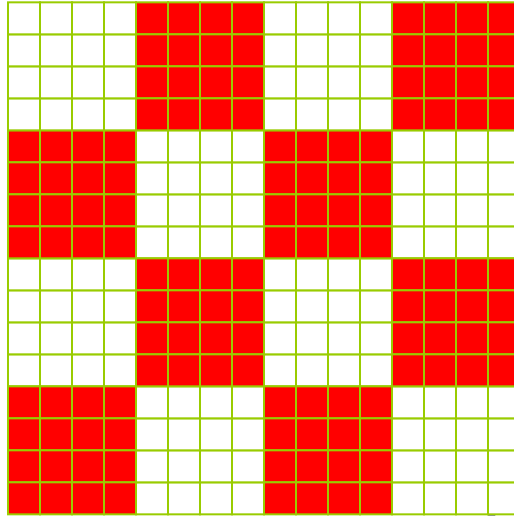
Mip Mapping

Quando e come si fa il rendering?

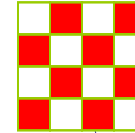
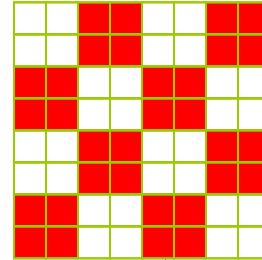
- Immaginiamo la texture ricoperta di pixel (cioè la dimensione dei pixel in texel nella texture originale)
- Si determini il livello del MIP Map in cui i texel sono approssimativamente in corrispondenza 1 ad 1 con i pixel
- Si interpolino i valori dei 4 texel più vicini



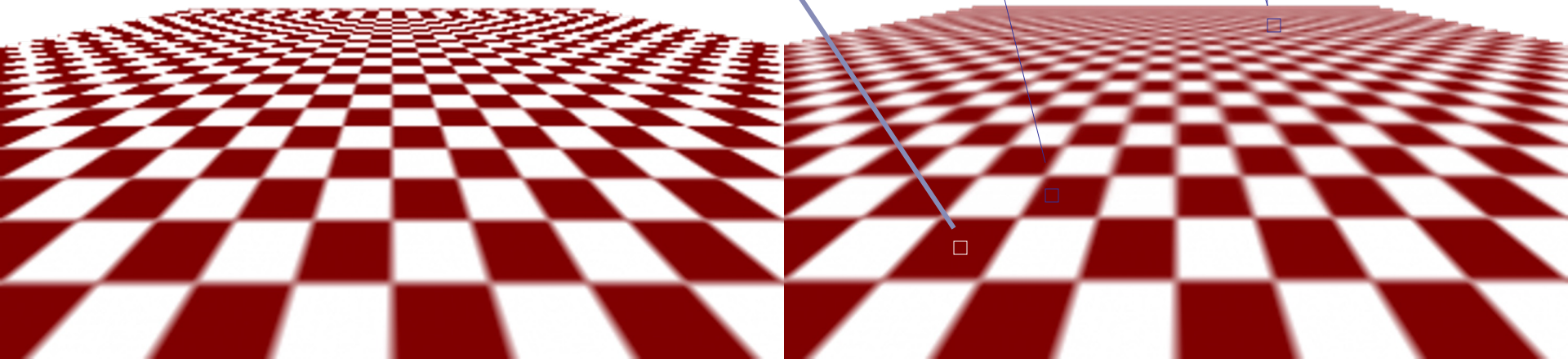
MIP Mapping: esempio

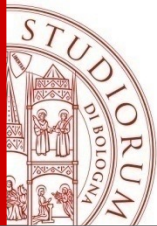


Bilinear Interpolation
non risolve il problema



MIP-Mapping



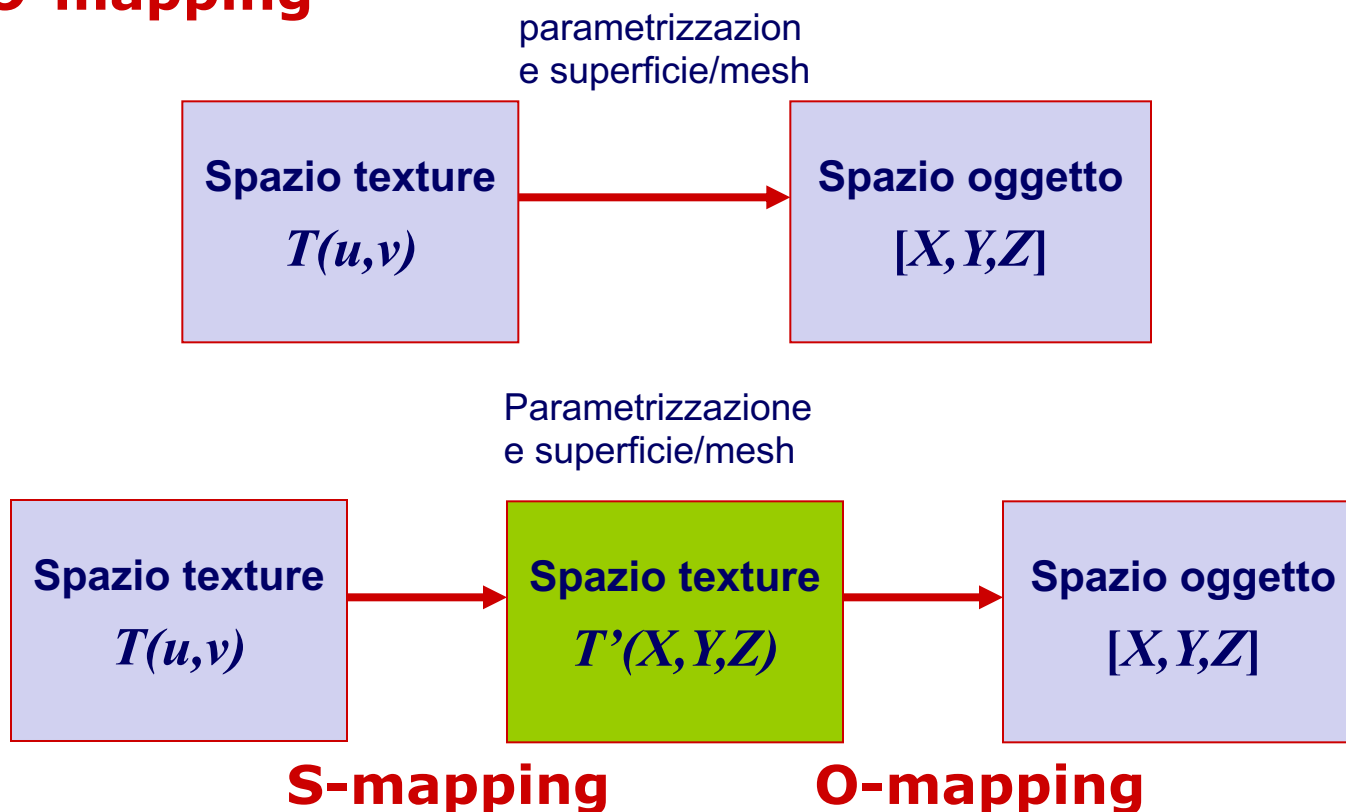


Texture Mapping 2D

Two-Part Mapping

1. S-mapping

2. O-mapping



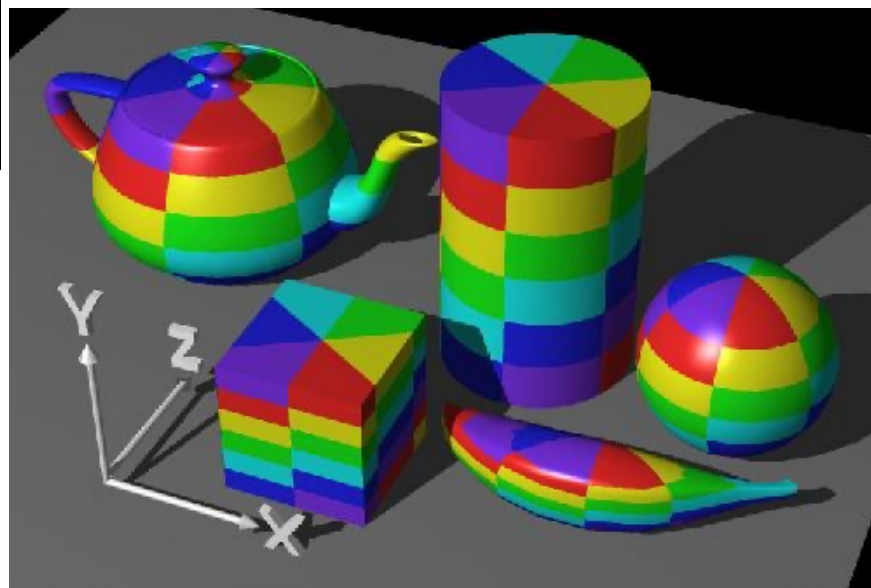
Texture Mapping 2D



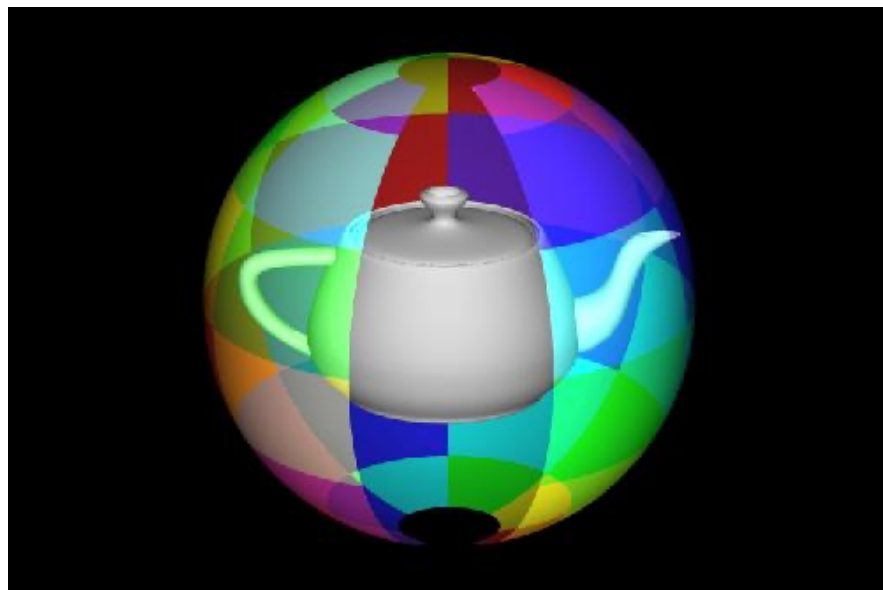
Superficie **S** intermedia:
Cilindro

Two-Part Mapping

1. S-mapping
2. O-mapping



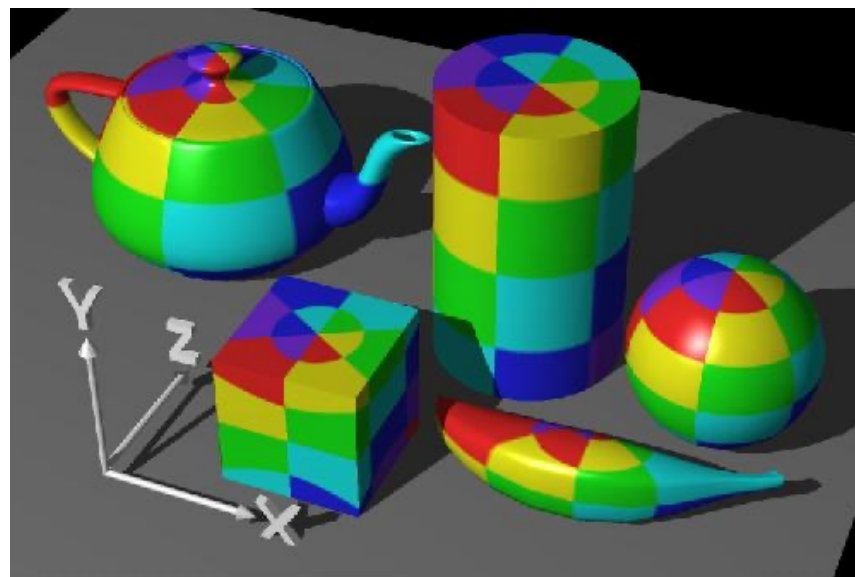
Texture Mapping 2D



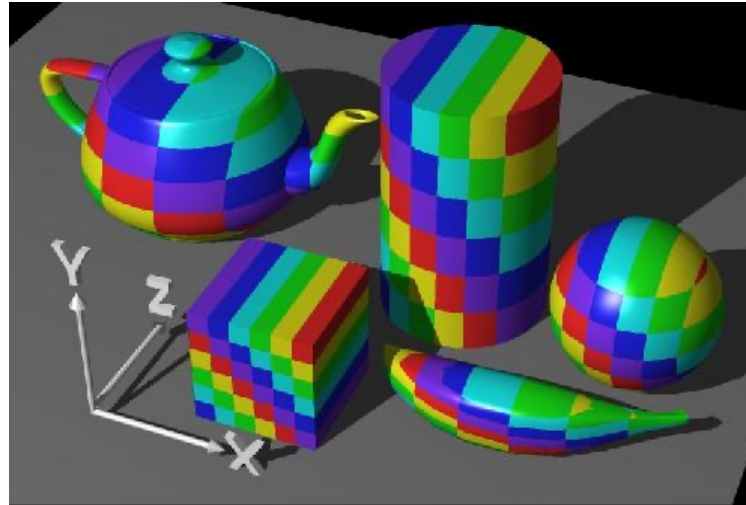
Superficie **S** intermedia:
Sfera

Two Part Mapping

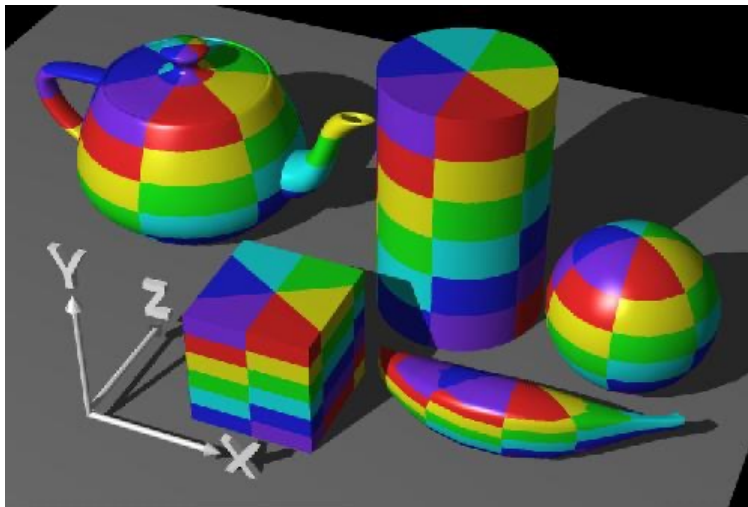
1. S-mapping
2. O-mapping



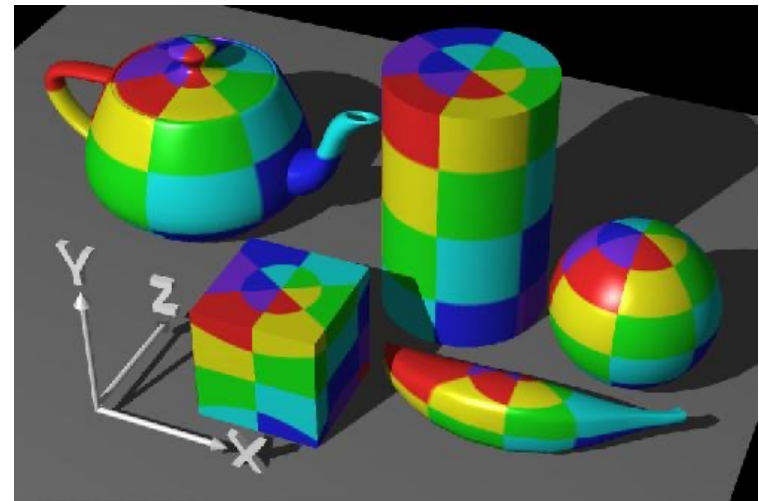
Texture Mapping 2D: confronto



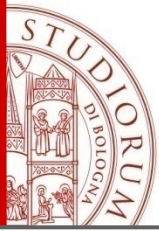
Piano XY



Cilindro



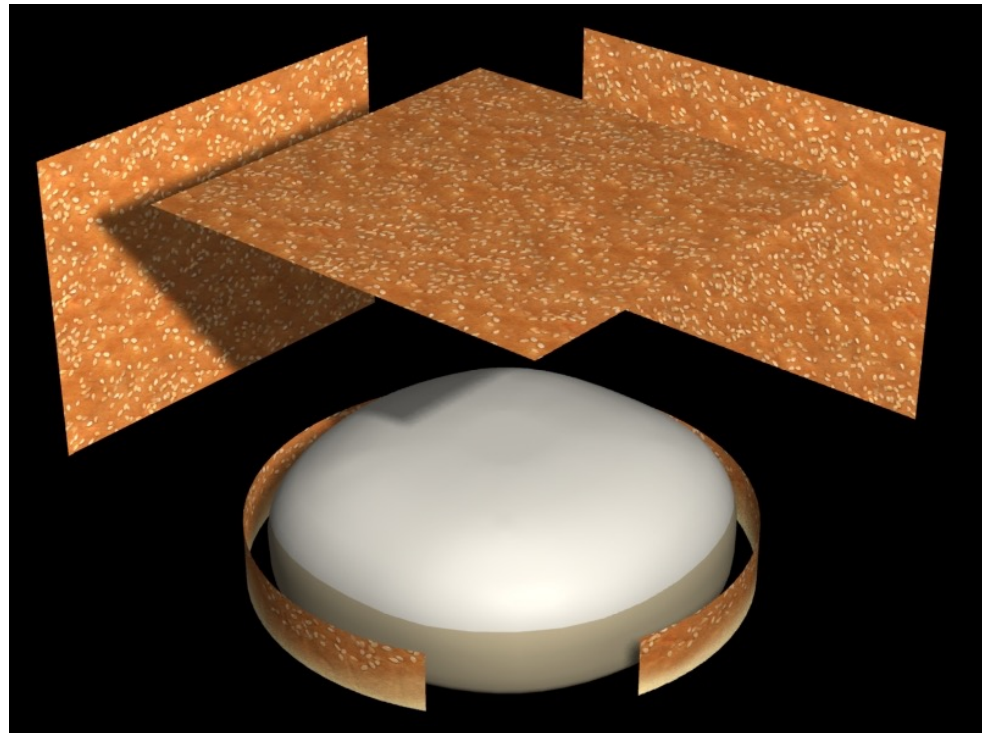
Sfera



Texture Mapping 2D: esempio



Two Part Mapping



Texture Mapping 2D: esempio



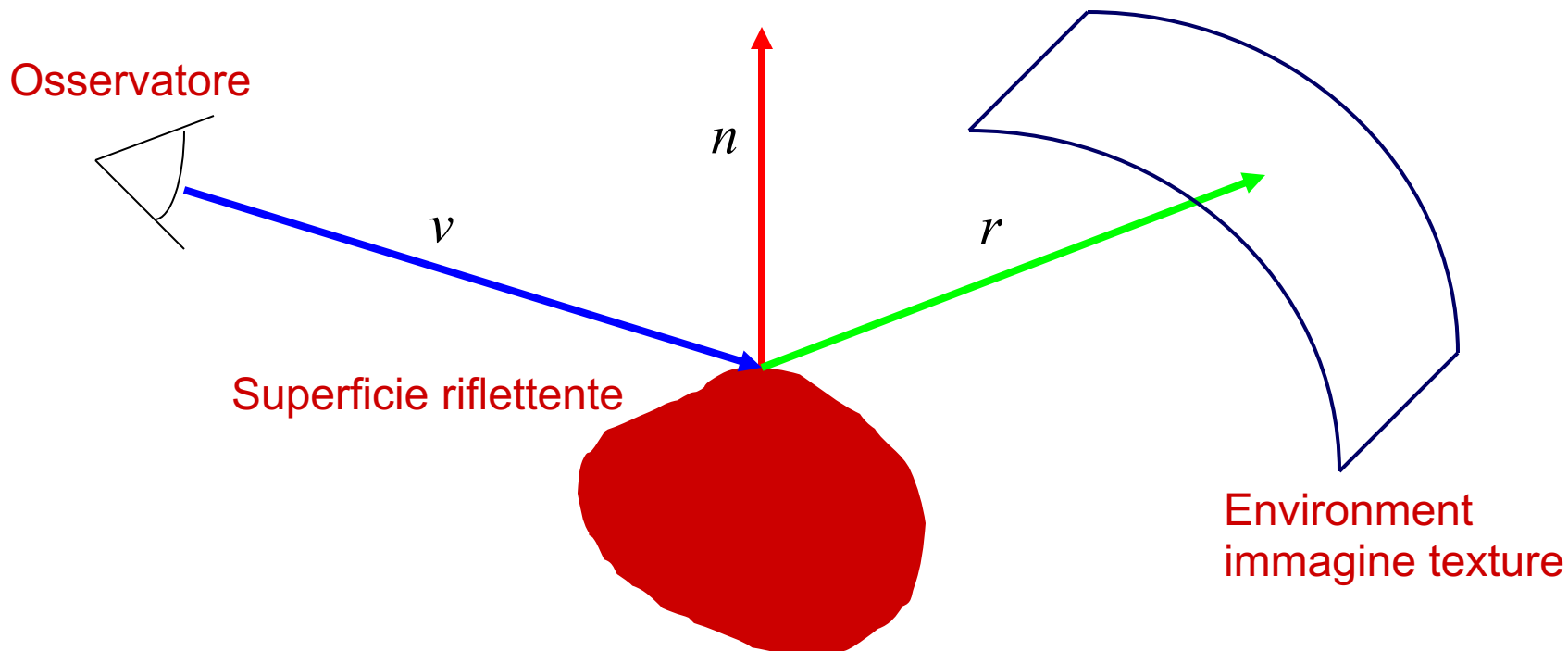
Texture Mapping 2D: esempio

Two Part Mapping:
cilindro



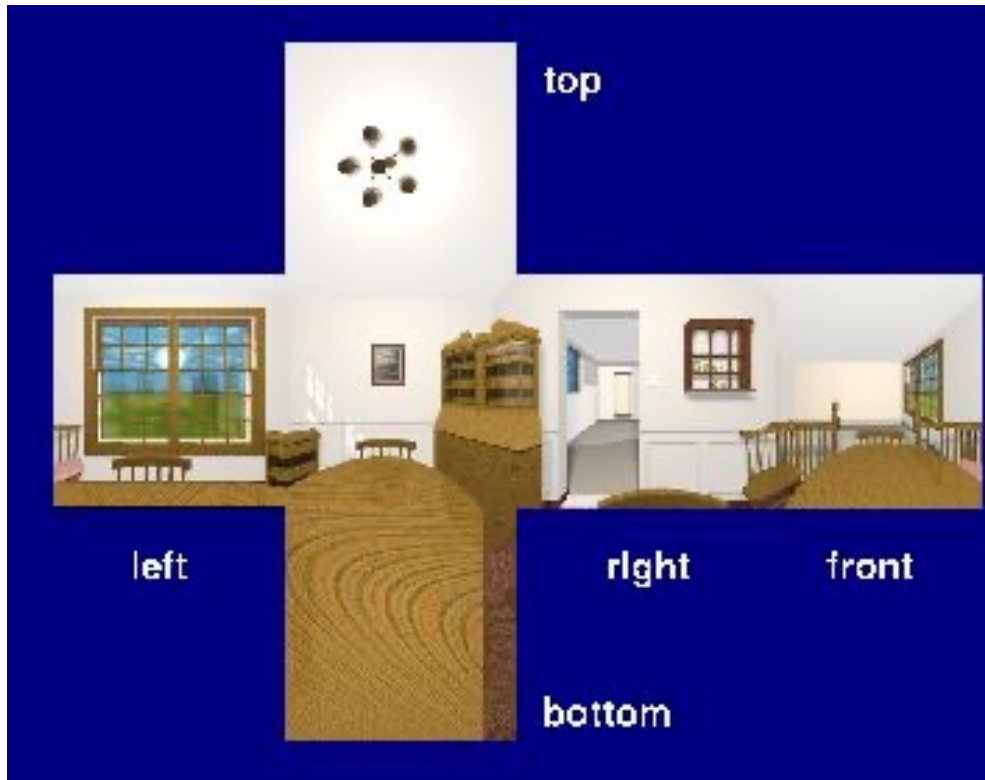
Environment Mapping

Raggio riflesso: $r = 2(n \cdot v)n - v$



La texture viene applicata nella direzione del raggio riflesso r , dall'environment immagine texture sull'oggetto

Cube Environment Mapping

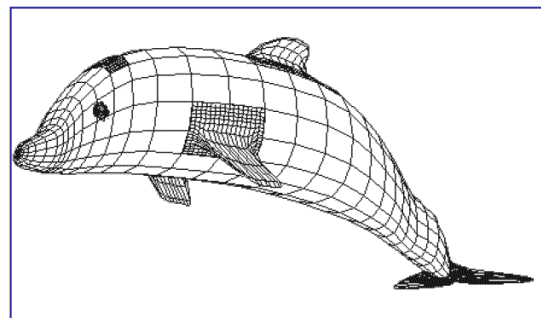
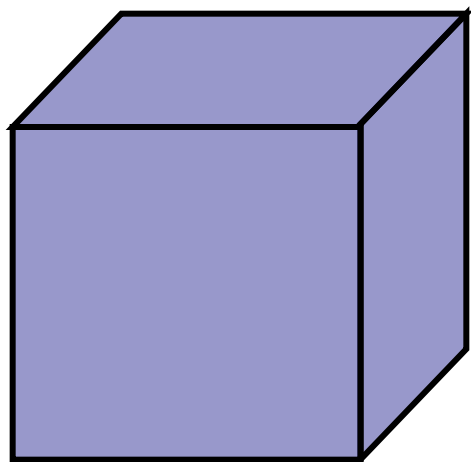


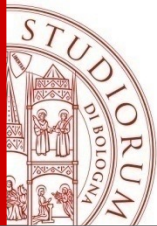
Solid Texturing

anche 3D Texture o Texture Mapping 3D o
Texture Procedurale

Si tratta di una texture a 3 dimensioni che viene applicata ad un modello 3D

L'oggetto 3D viene "immerso" nello spazio texture 3D e il colore di ogni punto dell'oggetto viene definito dalla sua posizione nello spazio texture.





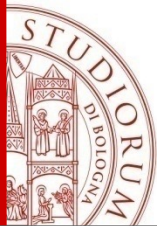
Solid Texturing

Vantaggi:

- Non è più necessario applicare texture 2D a modelli 3D
- Possibilità di simulare in maniera realistica materiali naturali (uso di texture procedurali 3D)
- Nuove possibilità di applicazioni

Svantaggi

- Alta occupazione di memoria delle texture (una texture 64x64x64 RGBA occupa 1MB, una texture 256x256x256 RGBA occupa 64MB)
- Per texture procedurali overhead computazionale che ne limita l'uso in applicazioni real-time (a meno che non siano state pre-calcolate)
- Difficoltà d'uso per l'applicazione all'oggetto di immagini (texture come immagine e non come procedural texture, es: volto di una persona)

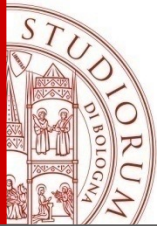


Solid Texturing: esempio



wood texture

Lo spazio 3D texture consiste in cilindri concentrici e alterni in colore; come un tronco d'albero.



Texture Procedurali

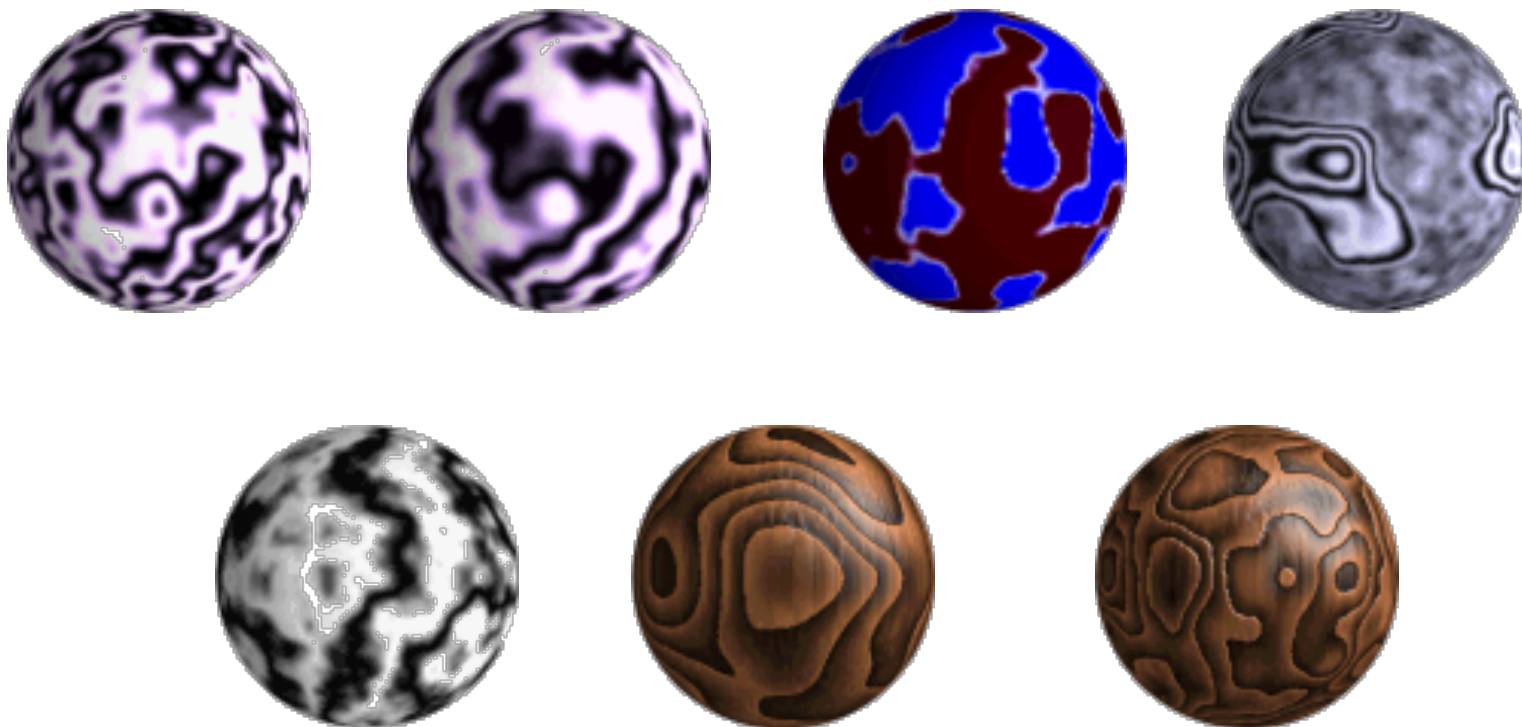
L'approccio **procedurale** non nasce con le texture 2D, ...
... ma trova il suo massimo sviluppo con le texture 3D.

E' possibile costruire texture procedurali anche 2D o 4D (dove la 4^a dimensione può essere il tempo – utile per creare effetti fisici realistiche, come le nuvole, ecc.).

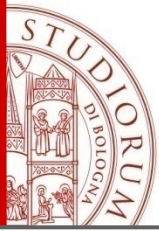
Nota: con metodo procedurale si intende un metodo matematico, tipicamente pseudo-casuale o di tipo frattale (sono anche detti **solid noise method**).

Questi metodi sono utili per la costruzione di texture che rappresentano materiali naturali come il legno, il marmo, la roccia, ecc.

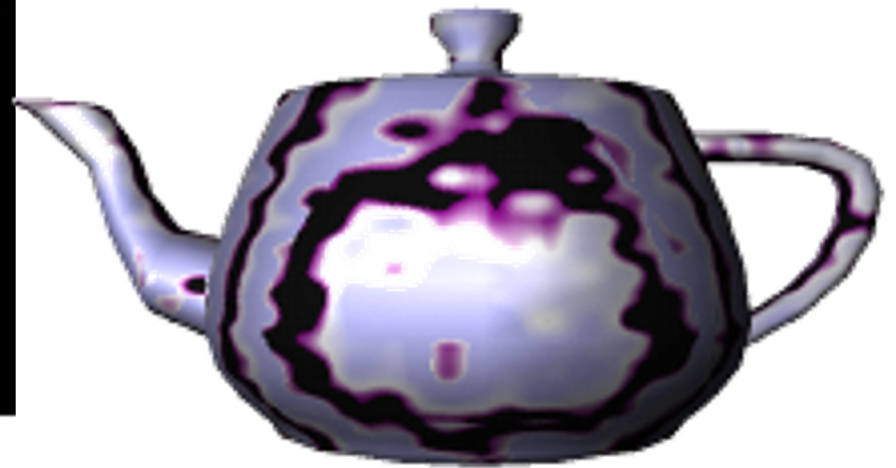
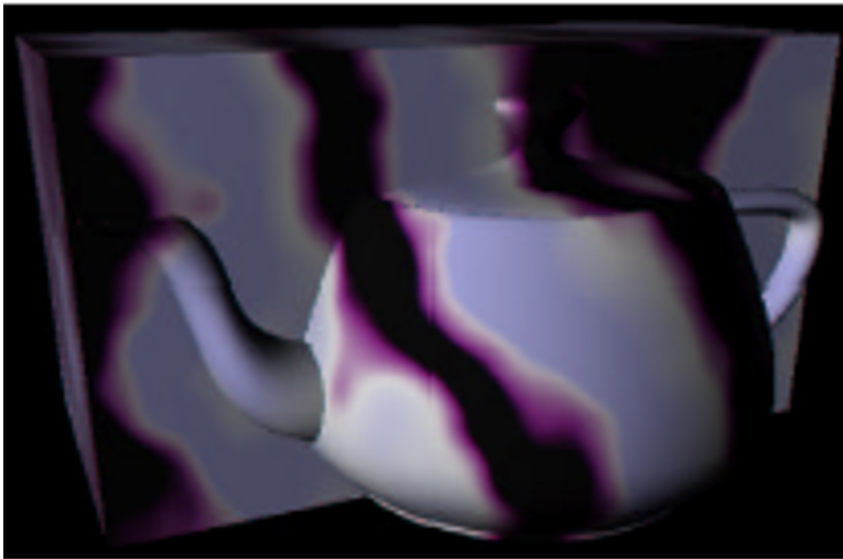
Texture Procedurali



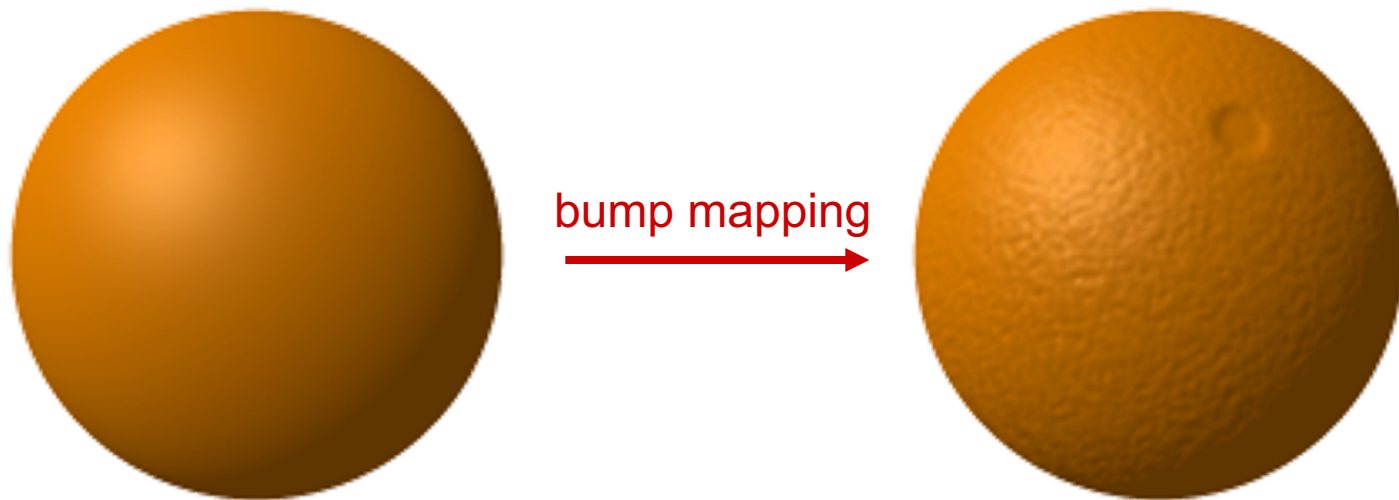
3D Perlin Noise



Texturing Procedurali: esempio



Bump/Normal Mapping



- Si sostituiscano i colori R, G, B con coordinate X, Y, Z
- Si interpretano i colori dei texel come vettori normali (Normal Map)
- L'immagine prodotta fa apparire la geometria molto più complessa di quello che è.

Bump Mapping

Blinn (1978): formulazione originale

Offset surface position



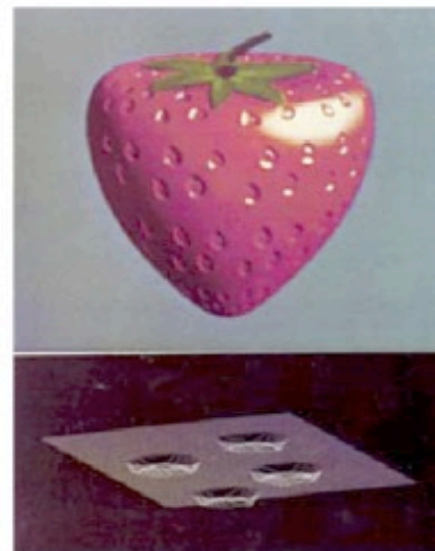
Displacement

$$\mathbf{N}(u, v) = \frac{\mathbf{P}_u(u, v) \times \mathbf{P}_v(u, v)}{|\mathbf{P}_u(u, v) \times \mathbf{P}_v(u, v)|}$$

$$\mathbf{P}'(u, v) = \mathbf{P}(u, v) + h(u, v)\mathbf{N}(u, v)$$

Perturb normal

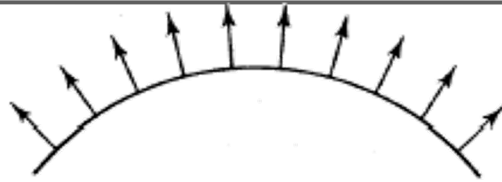
$$\mathbf{N}'(u, v) = \frac{\mathbf{P}'_u(u, v) \times \mathbf{P}'_v(u, v)}{|\mathbf{P}'_u(u, v) \times \mathbf{P}'_v(u, v)|}$$



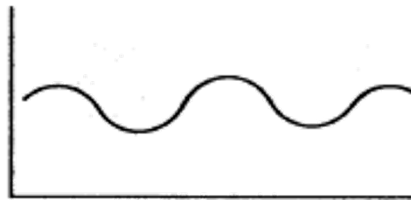
From Blinn 1978

Nel Bump Mapping originale (basato su una Height Map in scala di grigi), la formula richiede di calcolare il **gradiente** (la derivata) dell'altezza rispetto alle coordinate u e v per generare la perturbazione.

Bump Mapping



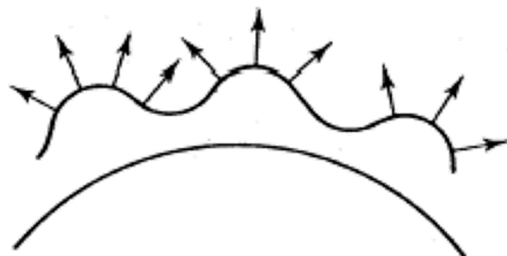
$O(u) \quad O(u,v)$
Original surface



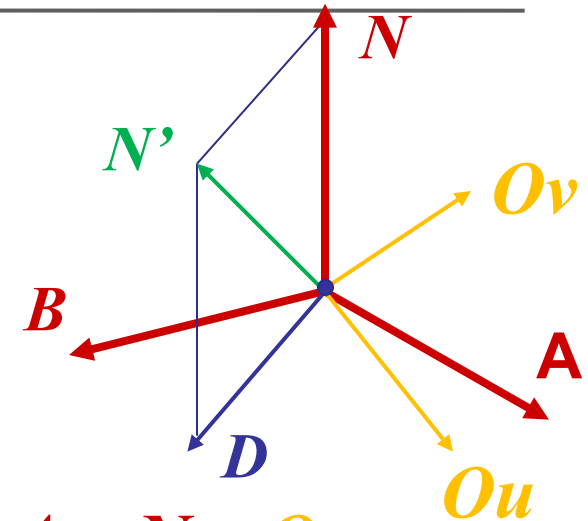
$B(u) \quad B(u,v)$
A bump map



$O'(u)$
Lengthening or shortening
 $O(u)$ using $B(u)$



$N'(u)$
The vectors to the
'new' surface



$$A = N \times Ov$$

$$B = N \times Ou$$

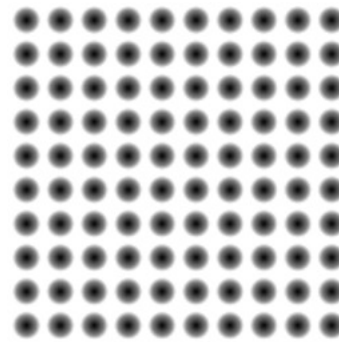
$$N' = N + D$$

dove $D = Bu \quad A - Bv \quad B$

$$Bu = dB(u,v)/du$$

$$Bv = dB(u,v)/dv$$

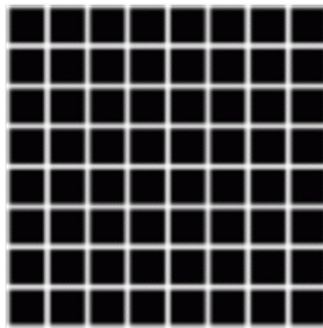
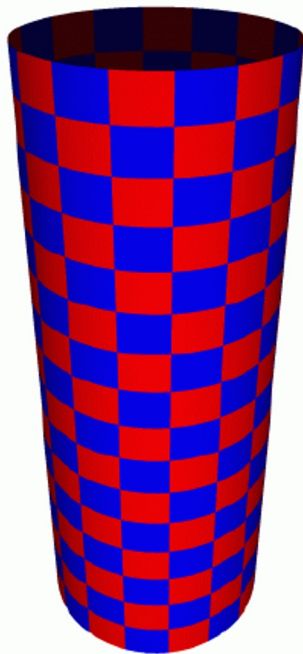
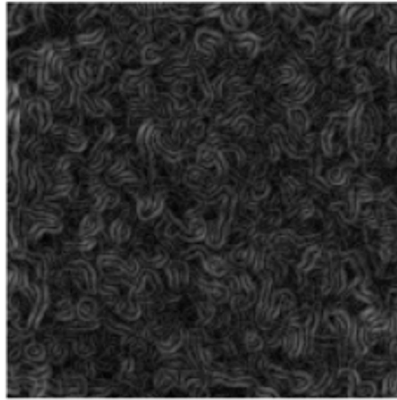
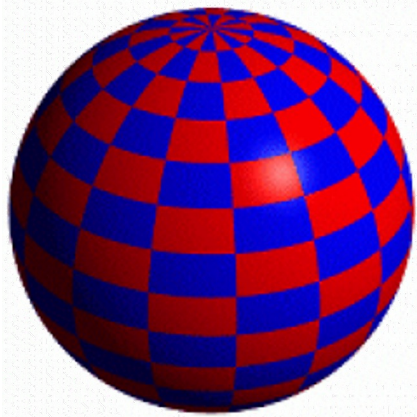
Bump Mapping: esempi



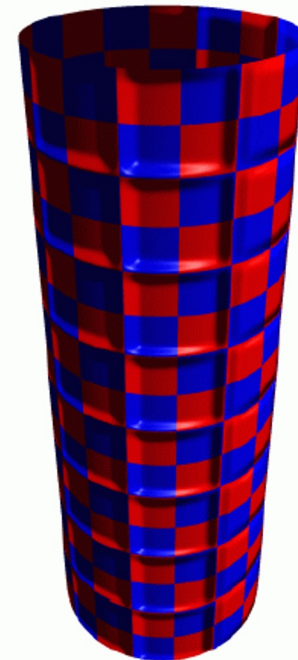
Bump Map



Bump Mapping: esempi



Bump Map



Bump Mapping: esempio



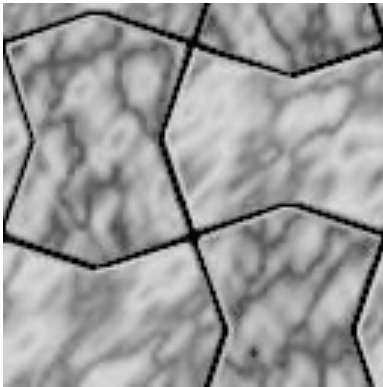
Bump Map



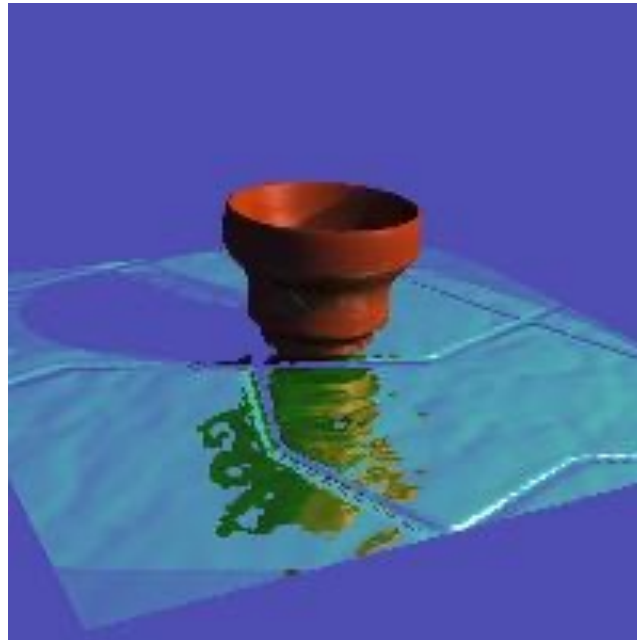
Bump Map sezione



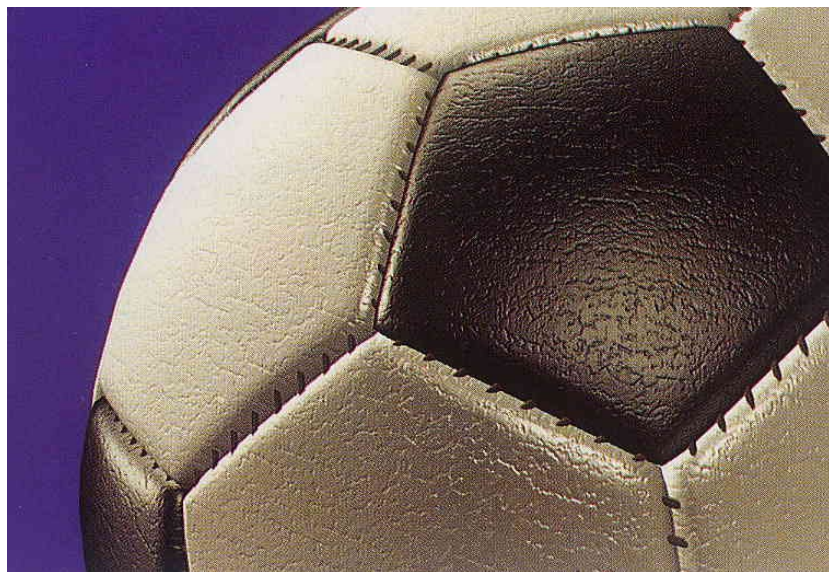
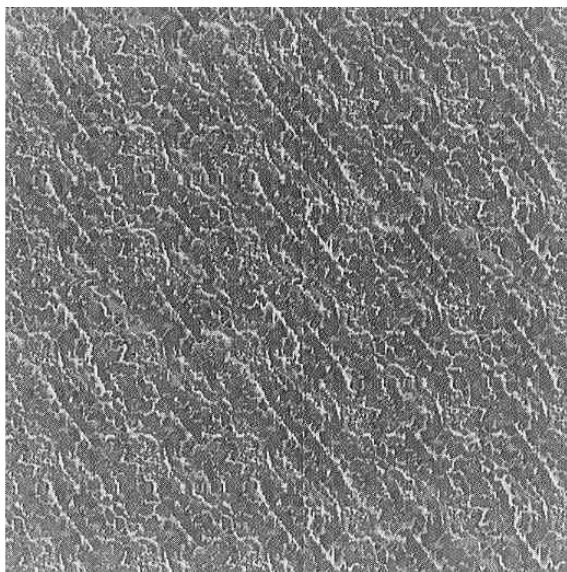
Bump Mapping: esempio



Bump Map



Bump Mapping: esempio

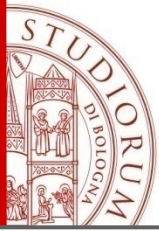


Bump Map

Multitexturing: esempio

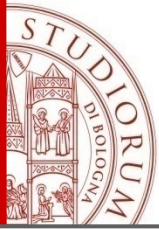


Texture Mapping 2D + Bump Mapping



Texture con WebGL

1. Associare le coordinate texture 2D ai vertici 3D della mesh
2. Specificare la texture
 - Leggere o generare l'immagine
 - Assegnare l'immagine alla texture RAM della GPU
3. Specificare i parametri della texture
 - Filtering, Wrapping, ...

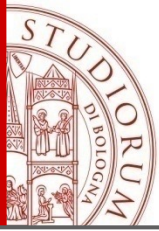


1. Associare le coordinate texture 2D ai vertici 3D

Le coordinate texture (coordinate 2D) sono attributi dei vertici della mesh e devono essere passati alla GPU come le coordinate dei vertici;

```
var texcoordBuffer = gl.createBuffer();  
gl.bindBuffer(gl.ARRAY_BUFFER, texcoordBuffer);  
  
gl.bufferData(gl.ARRAY_BUFFER,  
              new Float32Array(texcoord), gl.STATIC_DRAW);
```

Dove **texcoord** è un array JavaScript contenente le coordinate texture 2D per ogni vertice.

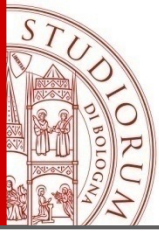


2. Creare un texture object e specificare la texture per quell'oggetto

Un texture object memorizza texture data (su GPU) e li rende disponibili per un loro utilizzo/riutilizzo

1. (`init( )`) Sia `image` l'id di un'immagine RGBA (array di texel);
2. (`init( )`) Crea un texture object
`texName = gl.CreateTexture( );`
3. (`init( )`) Associa un texture object ad un tipo di texture
`gl.bindTexture( target, texName );`

`target`: `gl.TEXTURE_2D`, `gl.TEXTURE_CUBE_MAP`



2. Creare un texture object e specificare la texture per quell'oggetto

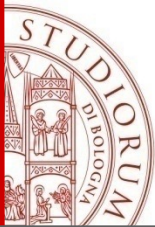
4. (**init()**) Copia un texture image 2D da un array di texel in texture RAM della GPU:

```
gl.texImage2D( target, level, internalformat,  
               w, h, border, format, type, image );
```

- **target**: gl.TEXTURE_2D, gl.TEXTURE_CUBE_MAP, gl.TEXTURE_CUBE_MAP_POSITIVE_X, ecc.
- **level**: livello del mipmap, 0 se non si utilizza mipmapping;
- **internalformat**: componenti colore (RGBA) considerate;
- w, h, border = width, height e bordo in pixel

Es.: `gl.texImage2D(gl.TEXTURE_2D, 0, gl.RGBA, 512, 512, 0, gl.RGBA, gl.UNSIGNED_BYTE, image );`

Anche: `gl.texImage2D(target, level, internalformat,
 format, type, image);`

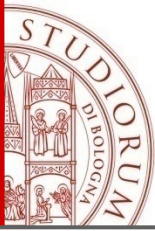


Texture e Shader (GLSL)

Come si è detto, ad ogni vertice viene associata una coppia di coordinate texture e ogni fragment/pixel ha un colore dato dall'interpolazione del colore dei **texel** associati ai vertici.

Le coordinate texture vengono lette insieme ai vertici dal Vertex Shader che le passa come varying/out al Fragment Shader che nella corrispondente variabile varying/out ottiene le coordinate interpolate (l'interpolazione viene effettuata dalla parte fissa della pipeline).

Quindi il Fragment Shader effettua il campionamento della texture in base alle coordinate texture interpolate.



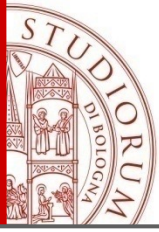
Texture e Shader (GLSL)

//Vertex Shader

```
attribute vec4 a_position;  
attribute vec2 a_texcoord;  
uniform mat4 u_matrix;  
varying vec2 v_texcoord;  
void main( ) {  
    gl_Position = u_matrix * a_position;  
    v_texcoord = a_texcoord;  
}
```

//Fragment Shader

```
precision mediump float;  
varying vec2 v_texcoord; // Passed in from the vertex shader  
uniform sampler2D u_texture; // The texture  
void main( ) {  
    gl_FragColor = texture2D(u_texture, v_texcoord);  
}
```



MIP Mapping

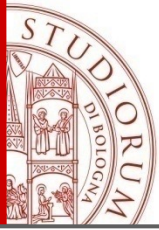
WebGL mette a disposizione il metodo:

```
gl.generateMipmap(target);
```

target: gl.TEXTURE_2D, gl.TEXTURE_CUBE_MAP

Un mipmap, come detto, è una collezione di immagini/
texture progressivamente più piccole, ciascuna di
dimensioni 1/4 della precedente.

La parte fissa della pipeline, una volta generate le texture,
se richiesto, sceglie quella che si adatta meglio alla
risoluzione delle facce da texturare.



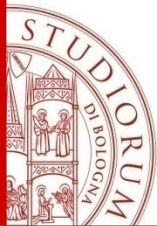
MIP Mapping e Bind

Attenzione: se le dimensioni delle immagini/texture non sono una potenza di due non si può richiedere a WebGL 1.0 di operare un MipMap.

E' bene procedere prima ad una **scala** dell'immagine/texture per portarla a dimensioni che siano potenze di 2

bind del texture object ogni volta che si deve rendere l'oggetto al quale la texture è associata

```
gl.bindTexture(gl.TEXTURE_2D, texName[i]);
```

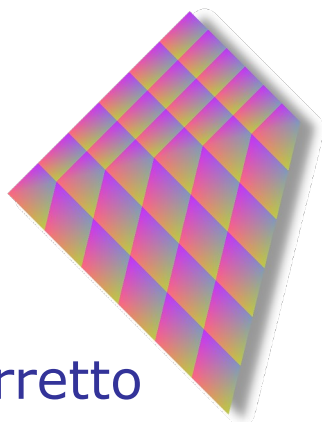


Interpolazione valori Texture

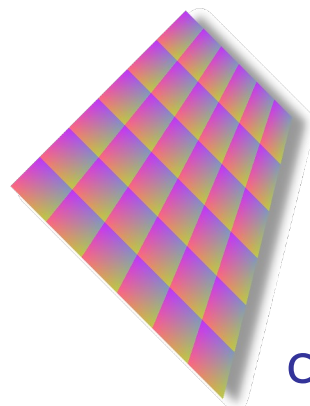
WebGL usa l'interpolazione bilineare per determinare le coordinate texture dei pixel delle facce triangolari così come per determinare i colori, dai colori dei vertici.

Per default l'interpolazione è con **correzione prospettiva**; a questo proposito si vedano i due codici nella cartella **HTML5_webgl_3**:

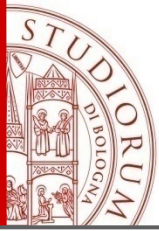
[cube_texture_perspective_non_correct.html](#)
[cube_texture_perspective_correct.html](#)



non corretto



corretto



Modalità di applicazione delle Texture

Si può chiedere a WebGL come gestire le texture settando per ognuna i parametri di filtering e di wrap, mediante:

```
gl.texParameteri/f( target , pname , ... );
```

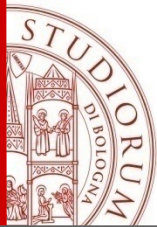
- **target:** gl.TEXTURE_2D, gl.TEXTURE_CUBE_MAP
- **pname:** gl.TEXTURE_{MIN,MAG}_FILTER
gl.TEXTURE_WRAP_{S,T},

Modalità di filtri

- minification o magnification
- speciali filtri di minification mipmap

Modalità Wrap

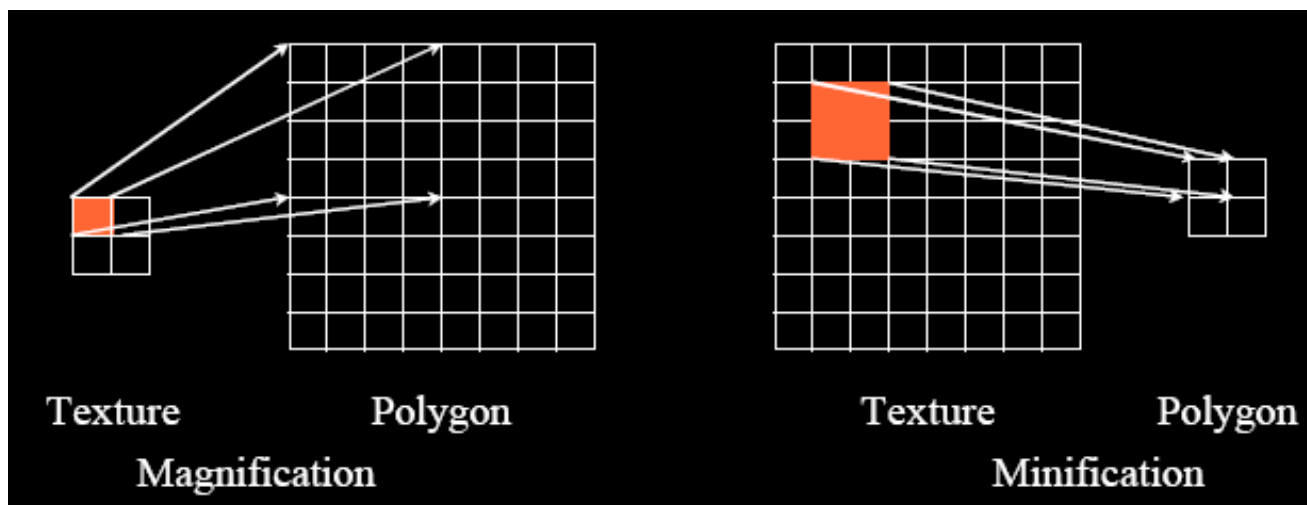
- clamping o repeating



Modalità di filtri: Anti-Aliasing

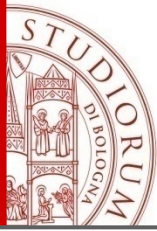
`gl.texParameteri( target, gl.TEXTURE_{MIN,MAG}_FILTER, param)`

param: `gl.NEAREST`, `gl.LINEAR`, `gl.NEAREST_MIPMAP_NEAREST`,
`gl.LINEAR_MIPMAP_NEAREST`, `gl.NEAREST_MIPMAP_LINEAR`,
`gl.LINEAR_MIPMAP_LINEAR`



Magnification:
il texel è più piccolo di un pixel
Soluzione: interpolazione

Minification:
il texel è più grande di un pixel
Soluzione: media

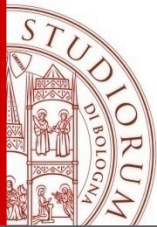


Modalità di filtri: Anti-Aliasing

`gl.texParameteri( target, gl.TEXTURE_{MIN,MAG}_FILTER, param)`

param: `gl.NEAREST`, `gl.LINEAR`, `gl.NEAREST_MIPMAP_NEAREST`,
`gl.LINEAR_MIPMAP_NEAREST`, `gl.NEAREST_MIPMAP_LINEAR`,
`gl.LINEAR_MIPMAP_LINEAR`

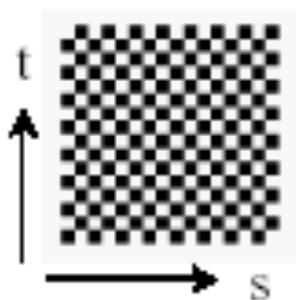
- `NEAREST` = scegli 1 pixel dal mipmap più grande
- `LINEAR` = scegli 4 pixel dal mipmap più grande e mediali
- `NEAREST_MIPMAP_NEAREST` = scegli il mipmap migliore, poi seleziona un pixel da quel mipmap
- `LINEAR_MIPMAP_NEAREST` = scegli il mipmap migliore, poi media i 4 pixel da quel mipmap
- `NEAREST_MIPMAP_LINEAR` = scegli i 2 mipmap migliori, scegli 1 pixel da ciascuno e mediali
- `LINEAR_MIPMAP_LINEAR` = scegli i 2 mipmap migliori, scegli 4 pixel da ciascuno e mediali



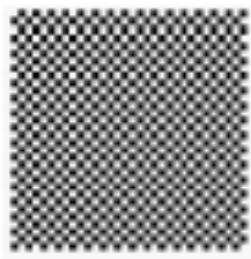
Modalità di Wrap

`gl.texParameteri( target, gl.TEXTURE_WRAP_{S,T} , param )`

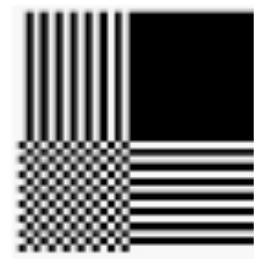
param: `gl.REPEAT`, `gl.CLAMP_TO_EDGE`, `gl.MIRRORED_REPEAT`



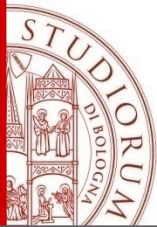
texture



`gl.REPEAT`
> [0,1]



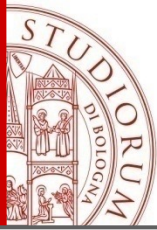
`gl.CLAMP_TO_EDGE`
Valori > 1.0 set 1.0
Valori < 0.0 set 0.0



Texture Mapping in WebGL

Nelle applicazioni è utile poter avere le coordinate texture in modo semplice/automatico per proiezione per esempio da un piano nello spazio, o mediante una superficie (two-part mapping), o altro, ma WebGL non mette a disposizione nulla di tutto ciò.

Sarà compito dell'utente programmare tali metodi o cercare librerie o software applicativi utili a questo scopo, come per esempio Blender.



Cube Environment Mapping

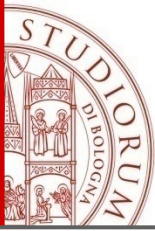
Come modalità automatica/hardware di applicazione delle texture, in WebGL esiste solo il **cube environment mapping**, cioè il mapping della texture sulla geometria specificata mediante riflessione sulle facce di un **cubo** che contiene la geometria stessa.

Questo si ottiene semplicemente specificando come target nelle varie chiamate

gl.TEXTURE_CUBE_MAP

e richiamando nel fragment shader la **function textureCube()**

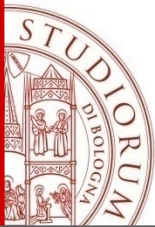
Nota: In WebGL 2, la funzione **textureCube** è stata unificata in **texture**, che capisce cosa fare in base al tipo di sampler.



Texture e Shader (GLSL)

//Vertex Shader

```
attribute vec4 a_position;  
attribute vec3 a_normal;  
uniform mat4 u_matrix;  
Uniform mat4 u_worldMatrix  
varying vec3 worldPosition;  
varying vec3 worldNormal;  
  
void main() {  
    // Multiply the position by the matrix.  
    gl_Position = u_matrix * a_position;  
    // send the world position to the fragment shader  
    worldPosition = (u_worldMatrix * a_position).xyz;  
    // orient the normals and pass to the fragment shader  
    worldNormal = mat3(u_worldMatrix) * a_normal;  
}
```



Texture e Shader (GLSL)

//Fragment Shader

precision highp float;

// Passed in from the vertex shader.

varying vec3 worldPosition;

varying vec3 worldNormal;

// The texture.

uniform samplerCube u_texture;

// The position of the camera

uniform vec3 worldCameraPosition;

void main() {

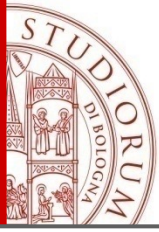
vec3 Normal = normalize(worldNormal);

vec3 eyeToSurfaceDir = normalize(worldPosition - worldCameraPosition);

vec3 direction = reflect(eyeToSurfaceDir, Normal);

gl_FragColor = textureCube(u_texture, direction);

}

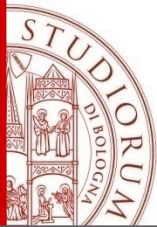


Esempi

Analizzare e provare i seguenti codici presenti nella cartella
HTML5_webgl_3

cube_textures_atlas.html e .js

cube_texture_environment_map.html e .js



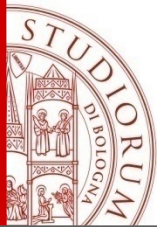
SkyBox ottimizzato

Invece di disegnare un cubo reale attorno alla scena e applicare su ogni faccia una texture, un modo ottimizzato per ottenere uno skyBox è quello di **disegnare un quadrato** che copre tutto il canvas e poi nel calcolare la **direzione** dello sguardo per ogni pixel sullo schermo e proiettarlo all'infinito, quindi "prendere" il colore corrispondente da una Cube Map.

Analizzare e provare i seguenti codici presenti nella cartella
HTML5_webgl_3

webgl_skybox.html

cube_texture_skybox_environment_map.html
(esempio con due shader program)



SkyBox ottimizzato: i passi

1. Trasformazione da Schermo a Spazio Mondo

Nel vertex shader `v_position` rappresenta i vertici del quadrato in Clip Space (coordinate da -1 a +1). Moltiplicandoli per l'inversa della matrice di proiezione e vista, si sta facendo il percorso opposto del rendering standard: si sta trasformando un punto 2D dello schermo in un vettore 3D che punta verso il mondo esterno.

2. Rimozione della Traslazione (la parte "ViewDirection")

La matrice non deve essere una semplice `viewInverse`, ma una `ViewDirectionInverse`. Cioè che dalla matrice di vista originale viene rimossa la traslazione (la posizione della telecamera).

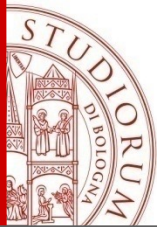
3. Risultato: Il Vettore di Direzione

Il risultato della moltiplicazione è un vettore che punta esattamente dal centro della camera a quel pixel specifico e entra nel "cielo/sky".

Questo vettore `v_direction` viene passato alla funzione

`textureCube(u_skybox, v_direction)`

per prelevare il colore corretto dalla cube texture `u_skybox`.



SkyBox ottimizzato

Moltiplicare per quella matrice serve a rispondere alla domanda:
"Se guardo il pixel (x,y) dello schermo, in quale direzione dello spazio 3D sto guardando?".

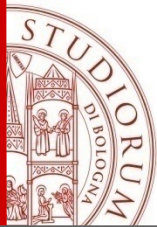
Il tocco finale di ottimizzazione:

Nel vertex shader, si imposta

```
gl_Position = vec4(a_position.xy, 1, 1);
```

Forzando la coordinata Z ad essere uguale a W , il sistema di coordinate normalizzate (NDC) vedrà $Z/W = 1$.

Questo garantisce che lo Skybox venga disegnato **esattamente sul piano di profondità più lontano**, venendo coperto automaticamente da qualsiasi oggetto della scena senza dover gestire manualmente i test di profondità.

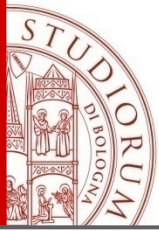


Texture e illuminazione

Se simuliamo l'illuminazione della scena e si vuole texturare un **oggetto** quali attributi o parametri devono essere modificati per produrre un effetto realistico?

I parametri più comunemente usati per il texture mapping sono i coefficienti di **illuminazione ambiente** e di **riflessione diffusa** (o colore) dell'oggetto, cioè **K_a** e **K_d** .

Ancora, come visto per il **bump-mapping**, si può agire sulla normale una cui perturbazione influenzerà la riflessione della luce producendo un effetto simile a quello di un materiale non liscio.



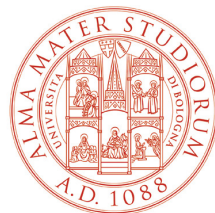
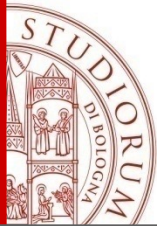
Codice Demo

Il bump-mapping è sperimentabile con il seguente codice Demo:

<https://math.hws.edu/graphicsbook/source/webgl/bumpmap.html>

I sorgenti di questa Demo sono scaricabili dalla pagina Web del corso:

bumpmap_demo.zip



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Giulio Casciola
Dip. di Matematica
giulio.casciola@unibo.it